

Notes on Neural Networks: part 3: Randomized approaches

Alessio Micheli

micheli@di.unipi.it

2024



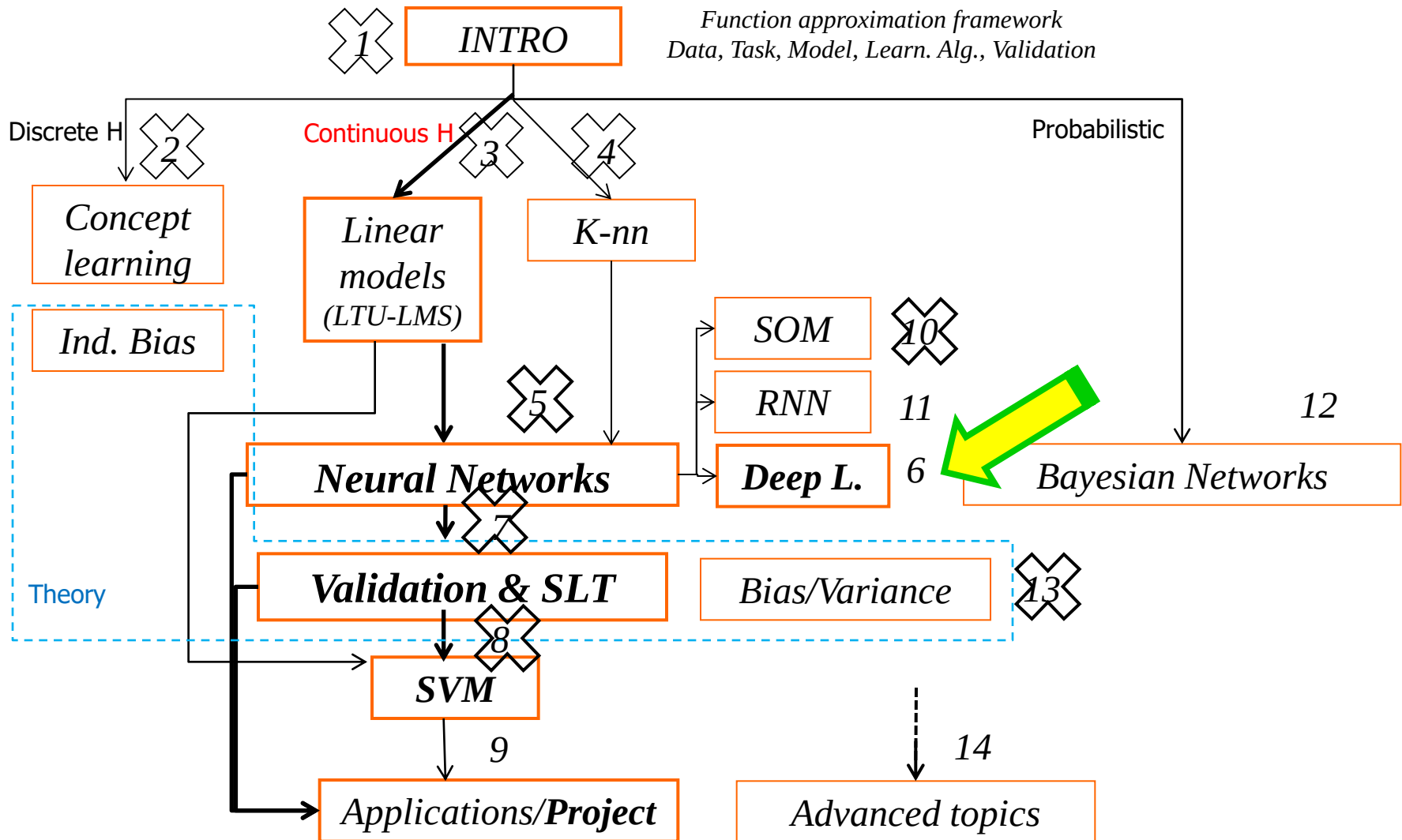
Dipartimento di Informatica
Università di Pisa - Italy

ML Course structure

Where we go



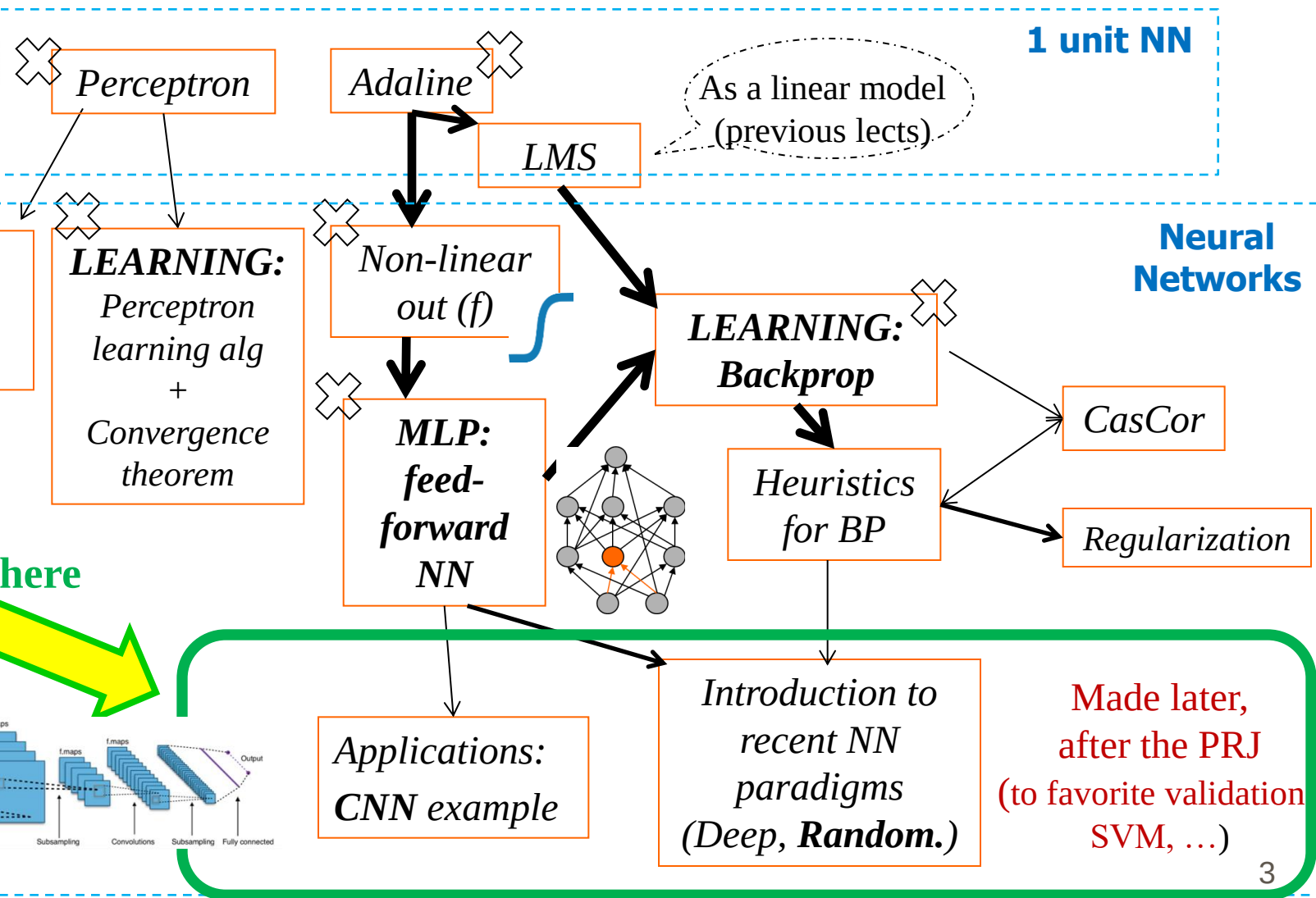
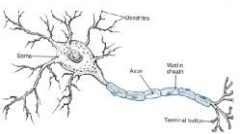
Dip. Informatica
University of Pisa



Course structure: zoom in the NNs box



Dip. Informatica
University of Pisa



Other architectures (guide slide)



Dip. Informatica
University of Pisa

Introduction to recent NN paradigms

- Other architectures.....
 - Deep learning
 - **Random weights NN**

AKA: *Deep* learning and *extreme* learning ;-)

NEW

Randomized Machine Learning & Random weights NNs



Introduction

A randomized approach employs a degree of randomness as part of its constructive strategy



- **Randomness** can be viewed as an **effective part** of **ML approaches**
 - Randomization can **enhance** several aspects of ML **methodologies**
 - data processing, learning, hyper-parameter selection
 - Randomization as a key factor for **building** efficient **ML models**
- **Focus:** Randomized Neural Network Approaches

Randomness in Machine Learning



Dip. Informatica
University of Pisa

Randomness can be present at different levels:

- To **enhance** predictive **performance**
- To **alleviate difficulties** of classical ML methodologies

Randomness in Specific Aspects of ML:

- Data Processing
- Learning and hyper-parametrization selection
- In the core design of the model

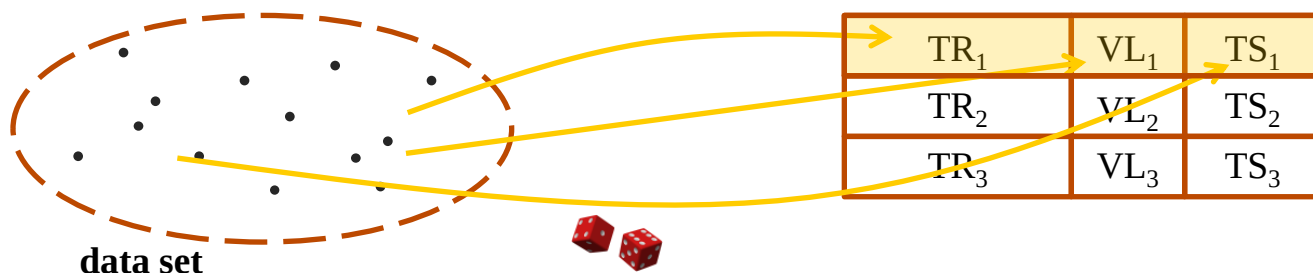
Randomness in Machine Learning



Dip. Informatica
University of Pisa

- **Data splitting**

- Hold-out, k-fold cross validation, leave-one-out



- **Data Generation**

- Bootstrap, Boosting, Balancing techniques

- **Learning:**

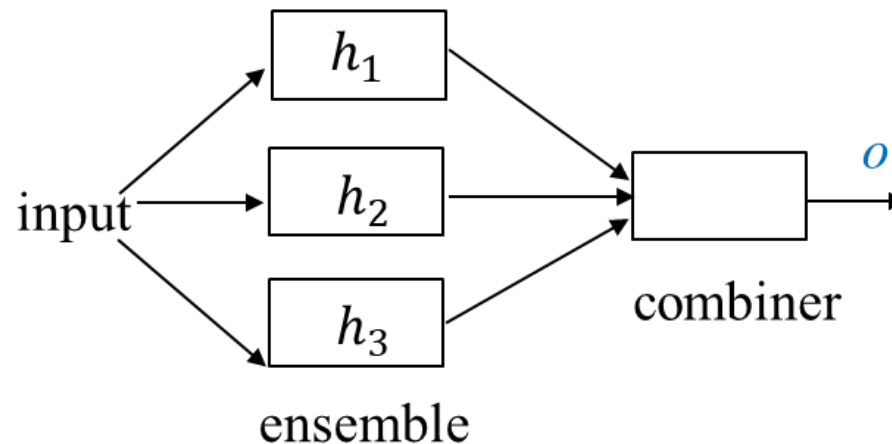
- e.g. observation order (shuffling for mini-batch)
- Random initialization of weights in NN
- ...

- **Hyper-parameters** selection (including random search)

- ...

Specifically: In the enesmbeling

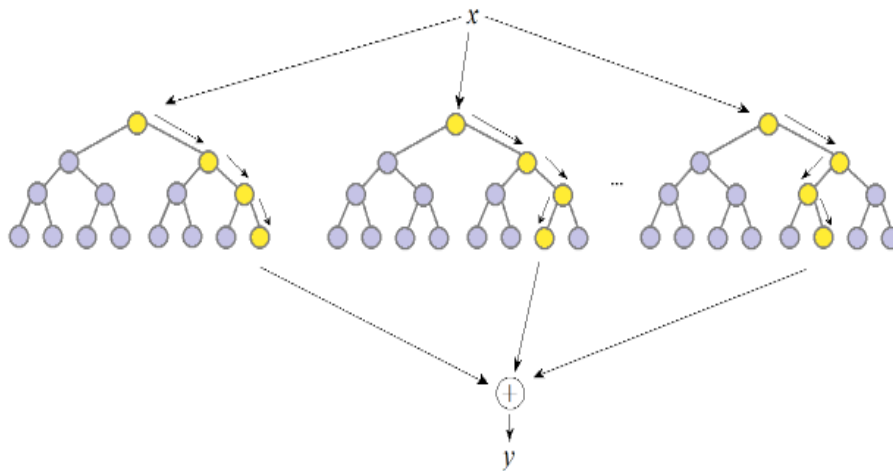
- Random differentiation among models, e.g.:
 - different weights init. in NN,
 - (partial) randomness for sampling different TR sets in bagging and boosting,
 - ...



Intrinsically Random Models

Randomness as a key element **of the ML model**

- Random neurons of Restricted Boltzmann Machines
- Extremely Randomized Trees, **Random Forests**



- Pool/ensemble of randomized decision trees
- Each Decision Tree is built from samples randomly drawn from the training set
- Randomized selection of the input variables to choose for splitting the nodes
- **Ensembling + Randomness advantages**

- Regularization: **Drop-out** for NN training (even if in this case randomization is not necessary per-se)

Random weights NNs: overview



Dip. Informatica
University of Pisa

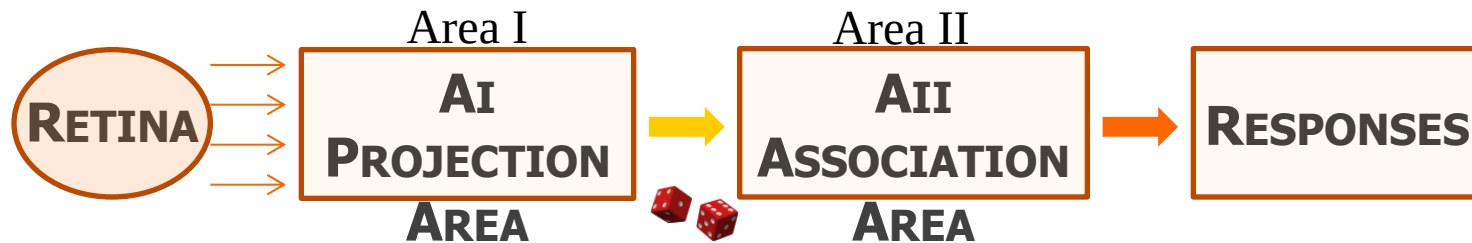
Is possible to exploit the **MLP/RNN architectures** even **without** long training cycles in the hidden layers?

- Long tradition for *randomized NN*, *random weights NN*, *Random projection architectures*, e.g. Random Vector Functional Link Neural Network (RVFL) ... since its origin
- Recently, new popularization with various names, including *ELM* (*Extreme Learning Machine*)
- Basic idea:
 - A network containing a **randomly** connected hidden layer(s), i.e. weights are **fixed** after random initialization
 - train **only** the output weights (linear model: by pseudoinverse/ridge regression)

overcoming the problems associated with the complex and computationally demanding training algorithms traditionally used to train neural networks.

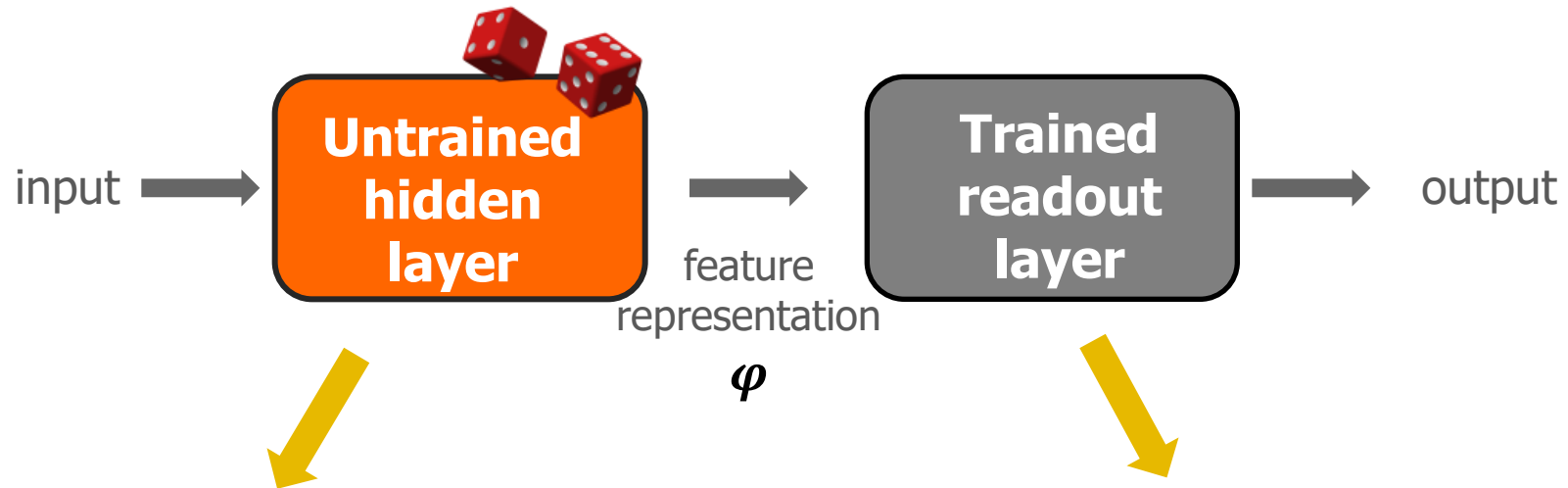
Randomized Neural Network Approaches

- In the Neural Network (NN) area randomness gave rise to a rich set of models
- **Roots are in the pioneering work on the Perceptron**



- Other pioneering works pointed out the **relevance of learning in the output layer** w.r.t. the hidden layer
- **Recently** randomized NNs delineate a **research line of growing interest**
- Particularly suitable when **efficiency** is of primary relevance (since training is restricted to the last layer!)

General Architectural Structure

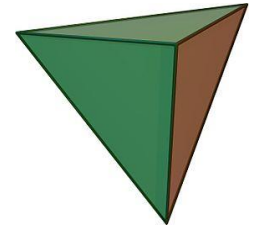


- **Non-linearly** embed the input into a **high-dimensional feature space** where the problem is more likely to be solved linearly (**LBE**)
- **Theoretically grounded** basis (Cover's Th. → next slide)
- Combine the features in the hidden space for **output computation**
- Typically implemented by linear models

Cover Theorem

- "A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated".

- In particular, using a deterministic mapping: Assuming l examples, lift them onto the vertices of the $l-1$ dim triangle (simplex).
Now, every binary partition of the samples is linearly separable

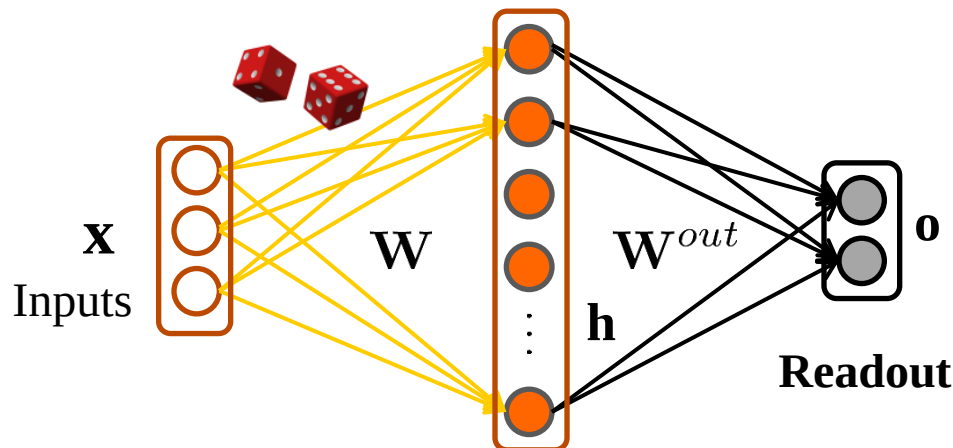


Cover, T.M., *Geometrical and Statistical properties of systems of linear inequalities with applications in pattern recognition*,
IEEE Transactions on Electronic Computers, **1965**.

- Note: The same property exploited for kernels !!!

Feed-forward Randomized NNs

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized large basis expansion** !



W hidden layer weight matrix

W^{out} readout weight matrix

f hidden layer non-linearity

TO BUILD:

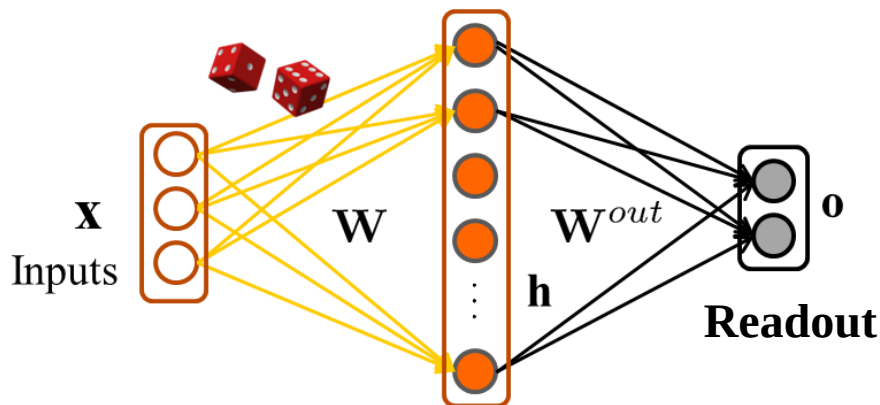
- Fill W with random values (e.g., Gaussian random noise) (using many units)
- Training for the Readout (see the next slide)

TO USE:

- Compute the hidden layer activations h as usual for neural units
- Compute the outputs o resulting in: $o(x) = W^{out} f(Wx)$
- Apply a threshold (or not linear functions if needed for classification)

Feed-forward Randomized NNs

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized basis expansion** !



W hidden layer weight matrix

W^{out} readout weight matrix

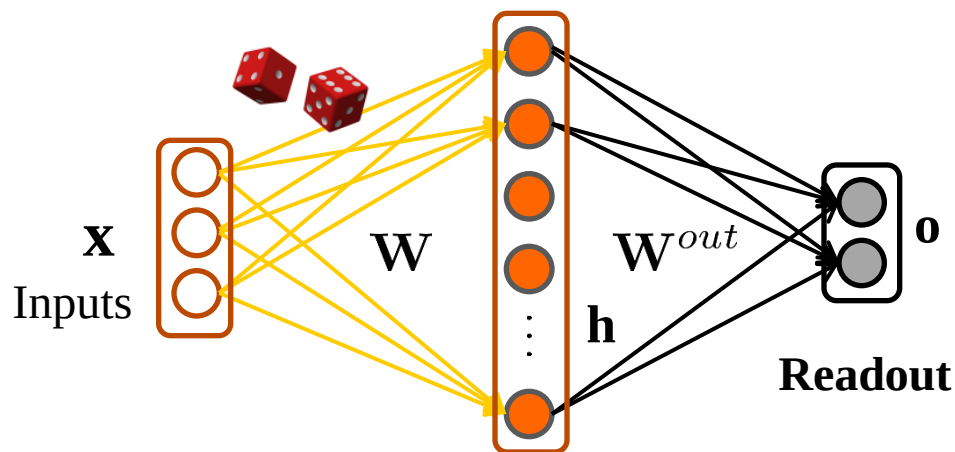
f hidden layer non-linearity

TO LEARN: given a set of l training example $(\mathbf{x}_p, \mathbf{d}_p)$, as we did for linear models: $\operatorname{argmin}_{\mathbf{w}} \operatorname{Error}(\mathbf{W}^{out})$ in L_2 by the direct method:

- Estimate W^{out} by least-squares fit (LMS) computed using the pseudoinverse: $W^{out} = H^+ \mathbf{d}$, where $H = f(WX)$
- OFTEN implemented with the L_2 regularization (useful due to the high number of units): $W^{out} = (H^T H + \lambda I)^{-1} H^T \mathbf{d}$
- or also by L1

Feed-forward Randomized NNs

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized basis expansion** (φ)



W hidden layer weight matrix

W^{out} readout weight matrix

f non-linearity of the expansion

Major reference literature models:

- Random Vector Functional-Link (**RVFL**)
- Extreme Learning Machine (**ELM**)
- Radial Basis Function (**RBF**) networks **with random centers**

Pro/Cons

- Large set of hidden unit plays an interesting role: projection non-linearly expands the dimension of an input vector so as to make the data more separable
- A large number of units can provide a sufficient "basis expansion" to solve the task at hand (with a theoretical ground, including universal approx. theorems)
- *Extremely **efficient***
- Many variants and improvement exists, e.g.:
 - incremental learning adding hidden neurons (see CC)
 - pruning, etc.They are often linked to past researches with a different name.
- Of course focused models, with smaller number of *adaptive* units (*basis functions*) trained on the specific task at hand appear still "more elegant" (there is a debate, but at least it is worth for you knowing also about this "random features" versus "learned features" approach)
- Extended to RNN → ESN (next lectures and CNS course)

Useful for

- **Big Data** (due to extreme efficiency)
- **Tiny implementations** suitable for distributed embedding e.g. in resource-limited devices, e.g. WSNs, IoT, edge computing...
- **Deep Learning**, where training algorithms are particularly demanding
 - Randomized NNs can **alleviate the computational burden** of training
 - **Deep Feed-forward Randomized NN**: Networks consisting in stacks of RVFL/ELM modules (e.g. TNNLS 2016)
 - Also randomized **CNN** approaches
- **Recurrent Randomized NN**
 - where efficiency of training is more demanding (wrt feedforward MLP)
 - with a solid ground on the introduced bias by system dynamic analysis (e.g. related to stability, learnability, etc.)
 - (we will introduce RNN in a next lecture)

Conclusions

- Randomness is not an enemy, it is a friend if you control it
- Also in other fields of CS (randomized algorithms is a research field), in Math (probability, also probabilistic ML approaches)
- Note that we are starting to see many times the same concepts (e.g. Cover theorem, LBE, regularization, feature representation, ...):
- The course is now «mature»

FAQ

- **Can ELM be used for the project?**

Yes, but as an additional part to the implementation of the standard MLP with SGD (fully trained with backprop). Easy to be implemented as a subpart of the fully trained FFNN/MLP.

More information

Examples (**ELM**):

- G.B. Huang, Q.Y. Zhu, C.K. Siew, "Extreme learning machine: theory and applications", *Neurocomputing*, vol. 70 (1), pp.. 489-501, 2006.
(just to have a reference, even if criticism on the mathematical part exists)
- G.B. Huang, Q.Y. Zhu, X. Ding, and R.Zhang. "Extreme Learning Machine for Regression and Multiclass Classification". *Trans. Sys. Man Cyber. Part B* 42, 2), pp. 513-529, 2012. (ELM + regularization (also with SVM))

Short Surveys:

- Y. Miche, B. Schrauwen, A. Lendasse, "Machine Learning Techniques based on Random Projections", *ESANN 2010 proceedings*, 295-302, 2010.
- C. Gallicchio, J.D. Martín-Guerrero, A. Micheli, E. Soria-Olivas, "Randomized Machine Learning Approaches: Recent Developments and Challenges" *ESANN 2017*, pp. 77-86

Recent Surveys:

- S. Scardapane, W. Dianhui. "Randomness in neural networks: an overview." *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 7.2 (2017): e1200.
- Le Zhang, P.N. Suganthan. "A survey of randomized algorithms for training neural networks." *Information Sciences* 364 (2016): 146-155.
- C. Gallicchio, S. Scardapane. "Deep randomized neural networks." *Recent Trends in Learning From Data* (2020): 43-68. Springer.

Bibliography (NN third part)

Please follow the many references trough the slides

Most of the parts from the DL book:

I. Goodfellow, Y. Bengio, A. Courville: ***Deep Learning***,
MIT Press, 2016 . On-line: <http://www.deeplearningbook.org/>

Q&A classes + Seminars

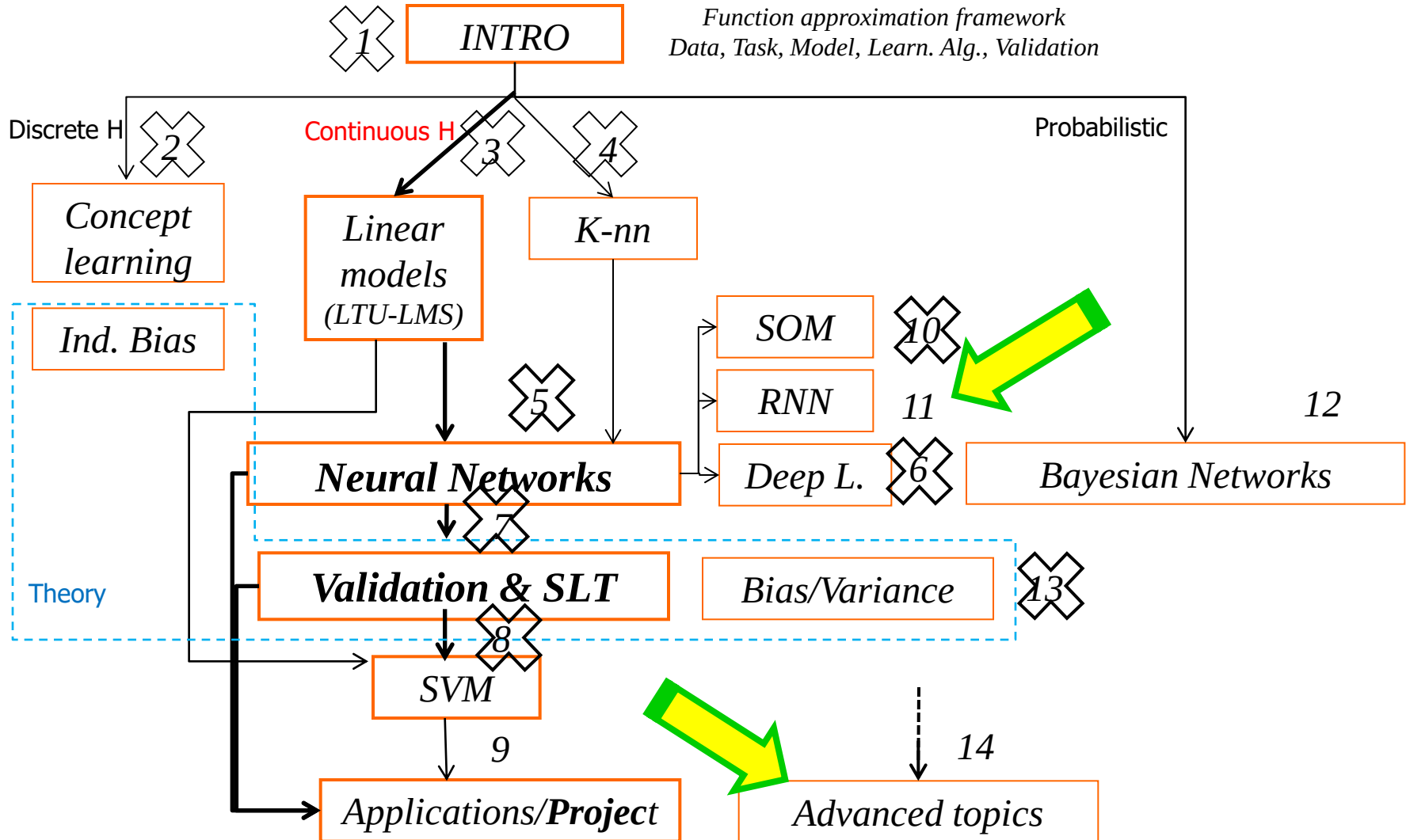
- Please, remember to send to me via email **questions** for the Q&A classes
 - To: micheli@di.unipi.it
 - Using the tag [ML-24-Q]
-
- Please remember to express your interest for possible **seminars** made by you (before the end of the course) (see lectures on DL)

ML Course structure

Where we go



Dip. Informatica
University of Pisa



Notes on Neural Networks: part 3

the end

Alessio Micheli

micheli@di.unipi.it

2024



Dipartimento di Informatica
Università di Pisa - Italy